Capstone Project

The AgTech Triathlon

From TurtleSim to a 'Real' Autonomous Tractor

The One-Sentence Pitch

You will deploy a physical Ackermann tractor (**MentorPi A1**) that autonomously drives a planned path across a **3 m** × **3 m** arena, dropping virtual seeds along the way, guided by an overhead ceiling-camera that acts as a *faux* RTK-GPS. Two optional stretch goals add weed spraying and dynamic geofencing for teams who finish Phase 1 with time to spare.

The Mission

The capstone is a **live deployment challenge**. You get as many **5-minute deployment windows** as time allows in the physical arena, but you must take turns with other groups via a queue. The ceiling camera tracks your robot that it makes decisions basedon, and your spacebar is the deadman.

The Real Goal: Get the Thing Moving

You have two full days to write your code and make it work: Wednesday is dedicated build time, and Thursday is the live deployment day where you take turns running your robot in the arena. The capstone passes when your robot drives the planned path autonomously, drops virtual seeds along the way, stays inside the arena, and obeys the spacebar deadman. That's it. That's the bar. Phase 2 (spray weeds at published coordinates) and Phase 3 (avoid a virtual wet zone on the return) are stretch goals to attempt only after Phase 1 is complete. They are worth additional marks, but they are not required to pass.

Phase 1 first. Get that working before you write a single line of Phase 2. Get Phase 2 working before you touch Phase 3. Do not try to write all three at once. Testing will be done on Wednesday but the official day for the runs are Thursday.

The God Node Architecture

The architecture is borrowed directly from RoboCup Small Size League and inverted from the Amazon Robotics warehouse model. Instead of putting an expensive sensor on every robot (*Inside-Out*), we put one camera on the ceiling and let it tell every robot where it is (*Outside-In*).

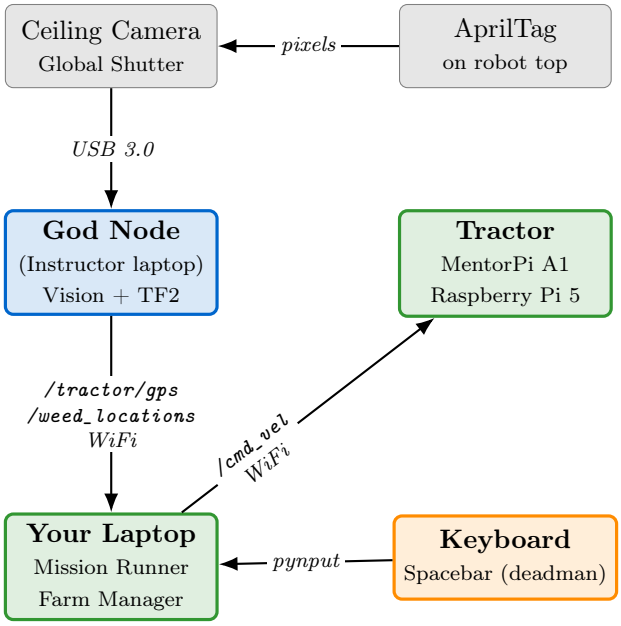

Figure 1: The Outside-In architecture. The God Node does all the heavy vision math and broadcasts position (and, for the stretch goal, weed locations). Your laptop runs the mission logic. The robot just obeys `/cmd_vel`. The spacebar holds veto power.

Why this split matters (Industry Parallel)

This is the same pattern used in autonomous warehouse robots (Amazon Kiva), surgical robots (Da Vinci), and many agricultural fleets (PTX Trimble). The robot is dumb on purpose; it just executes commands. The brains live on a more powerful machine, and the human always has the keys. You built every piece of this in the labs. Today you're just plugging them together.

The Arena

The physical field is taped onto the floor below the ceiling camera. The origin (0, 0) is marked with an X directly beneath the lens.

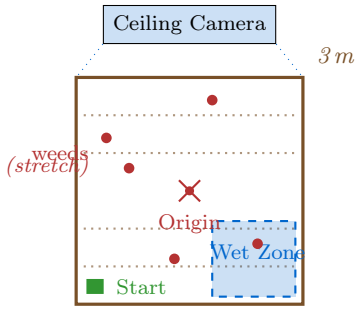

Figure 2: The 3 m × 3 m arena viewed from above. Origin is the red X on the floor. Crop rows (brown dotted) are 0.5 m apart. The Wet Zone and weed balls only matter if you're attempting the stretch goals.

Phase 0: Pre-Mission Safety Check

Gate to the Arena

Before any group puts a robot on the floor, they must demonstrate to the instructor that their deadman switch works.

This is the same Lab 10 safety test, run in front of the instructor. The robot stays on blocks (wheels free, not on the floor) for this check.

What you need to show:

1. Farm Manager and Mission Runner both running.
2. Spacebar **released**: `/cmd_vel` stays zero. Wheels do not move, even though the Mission Runner is publishing.
3. Spacebar **held**: wheels respond to the Mission Runner's commands.
4. Spacebar released mid-motion: wheels stop within 50 ms.
5. Kill the Mission Runner while holding spacebar: `/cmd_vel` drops to zero (auto-staleness check kicks in).

Once the instructor signs off, you may put the robot on the floor and join the queue for Phase 1.

Why this is non-negotiable

A robot driving autonomously without a working deadman is a hazard to itself, to other groups' robots, and to the people in the room. The deadman demo takes less than a minute. The cost of skipping it could be a damaged robot, a damaged person, or both.

Phase 1: The Main Mission (Seeding)

What You Have to Do

Drive a pattern coverage path across the field. Every 0.10 m of travel, fire a seed-drop trigger so the instructor's RViz logs a seed at your robot's current position. Stay inside the arena, since the AprilTag detection does not work outside the camera view. The spacebar must hold absolute veto authority over motion.

The seed-drop trigger. When your Mission Runner has travelled 0.10 m since the last seed, publish an empty message on:

```
/seed_drop  (std_msgs/Empty)
```

The instructor's God Node listens for this, grabs the current robot pose from the ceiling camera, and publishes a `visualization_msgs/Marker` on `/seeds` which is displayed live in RViz. You decide *when* to drop a seed. The God Node decides *where* it lands. RViz shows the truth.

Pass condition:

- Robot drives the planned path from start to end autonomously.
- Seed-drop triggers fired at roughly 0.10 m intervals along the path.
- Marker pattern in RViz roughly traces the planned path (this is how we grade "did the robot actually drive where you said it would").
- No deadman violations (releasing the spacebar always stops the robot within 50 ms).

Strategy: Do *not* use time to trigger seeds, your robot will drift and slip or enter turns (do not seed the headland). Compute the Euclidean distance from your current pose to the location of the last dropped seed. When that distance is at least 0.1 m, fire the seed-drop service.

Skills you already have:

- **Lab 4** (Services and Composition). The seeder pattern with `add_done_callback` so the async service call doesn't deadlock your control loop. You already wrote `auto_seeder.py` and `key_seeder.py`; the capstone seeder is a distance-triggered variant.
- **Lab 7** (Autosteer). Generating the snake path using linear interpolation between row endpoints and parametric arcs for the headland turns.
- **Lab 9** (God Node Bridge). Subscribing to `/tractor/gps` instead of `/turtle1/pose` as your position source.
- **Lab 10** (Farm Manager). The complete safety stack. Your Mission Runner publishes to `/auto/cmd_vel`, and the Farm Manager gates everything through the spacebar deadman.

Phase 2: Weed Spraying (Stretch Goal)

If Phase 1 is solid

The instructor scatters red balls in the arena and broadcasts their coordinates on a ROS topic. Subscribe, navigate to each, stop within ±10 cm, and "spray" (hold zero velocity for 3 seconds while logging).

The weed feed. The God Node publishes the list of weed locations on:

```
/weed_locations  (geometry_msgs/PoseArray)
```

The message is published with `transient_local` (latched) QoS, so even if your subscriber starts late, you still get the most recent list. Each pose's (x, y) is the centre of a red ball in arena coordinates.

Why the God Node hands you coordinates: the God Node already has perfect vision of the arena and runs the weed-detection pipeline. It would be redundant for every robot to also run vision. This is the same logical split as in self-driving fleets: centralized perception, distributed control. Your job here is the *navigation*.

Planning the Visit Order

You receive a list of (x, y) points. You need to decide what order to visit them in, then connect them with a drivable path. Two things to think about:

Order: nearest-neighbour is fine. Optimal route-planning (the Travelling Salesman Problem) is NP-hard and not worth your time. A greedy nearest-neighbour walk gets you 90% of the way for almost no code:

```
def nearest_neighbour_order(start, weeds):
    """Return weeds in greedy nearest-neighbour order from start."""
    remaining = list(weeds)
    order = []
    cursor = start
    while remaining:
        nxt = min(remaining, key=lambda w: math.hypot(w[0]-cursor[0],
                                                      w[1]-cursor[1]))
        order.append(nxt)
        remaining.remove(nxt)
        cursor = nxt
    return order
```

Path between weeds: do not just point the wheel and go. On an Ackermann robot, a straight-line "head toward target" controller will overshoot every weed and oscillate, because the robot cannot turn in place. You already solved this problem in Lab 10 with Pure Pursuit. Reuse it. For each weed:

1. Generate a short breadcrumb path from your current position to the weed (same linear interpolation trick from Lab 7).
2. Feed those breadcrumbs into your Mission Runner's Pure Pursuit follower.
3. When your distance to the weed drops below 0.10 m, transition to `SPRAYING`.

Densifying breadcrumbs between points. Reuse your Lab 7 code:

```
def breadcrumbs_to(start, goal, step=0.05):
    """Return (x, y) breadcrumbs from start to goal, spaced ~step metres."""
    dx, dy = goal[0]-start[0], goal[1]-start[1]
    dist = math.hypot(dx, dy)
    n = max(1, int(dist / step))
    return [(start[0] + dx * i/n, start[1] + dy * i/n) for i in range(n+1)]
```

Strategy: On startup, take the list of weeds from `/weed_locations`, run nearest-neighbour to get a visit order, and treat each leg as a mini-mission. Your Pure Pursuit follower doesn't care whether its breadcrumbs come from a snake-pattern generator or from a leg-to-the-next-weed generator. The path is the path.

Approach Angle Matters

The robot cannot suddenly stop on a dime, and it cannot approach a weed from any direction it wants. If your last breadcrumb sits exactly on the weed, the robot will overshoot before its control loop catches up. A small fix: end your breadcrumb path 5–10 cm *short* of the weed and let momentum carry it to the target. Then transition to `SPRAYING` based on distance to the weed, not breadcrumb-index.

Phase 3: Wet-Zone Avoidance (Stretch Goal)

If Phase 2 also works

Return to base. Mid-mission, the God Node will broadcast the coordinates of a virtual "wet zone" rectangle that you must **not enter**. Pass condition: return to the origin without ever entering the geofence.

Strategy: Subscribe to the wet-zone polygon from the God Node. Before publishing each `/auto/cmd_vel`, project the next waypoint onto the path and check whether the segment intersects the polygon. If it does, insert a temporary detour waypoint along the perimeter. You should re-planning a path around the zone instead of just stopping.

Skills you already have:

- **Lab 3** (State Machines). Extend the mission state machine with an `AVOIDING_WET_ZONE` state.
- **Lab 7** (Path Planning). Reuse breadcrumb generation to densify a detour path around the polygon perimeter.

Sim-to-Real: The Reality Gap

You are about to confront a phenomenon that every person with works with robots meets the first time they leave the simulator: *it worked on my laptop, why is it crashing?*

The "Death Wobble" Warning

TurtleSim is a frictionless point mass. The real MentorPi has momentum, an Ackermann turning radius of about 0.35 m, motor latency, encoder noise, and Wi-Fi packet drops. If your physical robot tries to correct its heading instantly at maximum speed, it will suffer violent oscillation (the "Death Wobble") and crash out of bounds.

You cannot cheat physics. You can predict latency, dampen overcorrection, and slow down on turns.

The total latency budget for your control loop is roughly:

$$t_{\text{total}} = \underbrace{t_{\text{cap}}}_{\text{frame interval}} + \underbrace{t_{\text{proc}}}_{\text{vision math}} + \underbrace{t_{\text{net}}}_{\text{WiFi}} + \underbrace{t_{\text{wait}}}_{\text{control cycle}} \approx 60\,\text{ms}$$

At a cruise speed of 15 cm/s, that's already about 9 mm of positional drift built into every cycle. You will not get sub-centimetre precision; design accordingly.

Three Reality-Gap Bugs You Will Hit

1. **Yaw overcounting.** As you discovered in **Lab 8**, the EKF on the MentorPi overcounts rotation: a 45° physical turn reports as more than 2.3 rad. Do not trust raw IMU yaw for turn termination. Use a calibrated timer or the God Node's θ.
2. **The robot is not the turtle.** TurtleSim updates instantly; the real robot has a delay between when you publish a command and when the wheels respond. If you tuned aggressive control gains in sim, expect overshoot on the real robot. Start with slower speeds and gentler turns than your sim settings.
3. **Ackermann constraint.** **Lab 3** taught you this in simulation. On the real robot, `linear.x = 0.0; angular.z = 1.0` does nothing except burn out the steering servo. Every turn must be an arc: `linear.x > 0`.

Required Skills Inventory

Every skill below was earned in a specific lab. If any of these feel shaky, revisit that lab *before* the dress rehearsal.

Lab	Skill	Where it's used in the capstone
Lab 2	Publishers, timers, OOP nodes	Every controller you write; the 20 Hz control loop.
Lab 3	Subscribers, callbacks, state machines	The mission state machine; transitions between path-follow and seed-drop.
Lab 4	Services, clients, async patterns	Dropping virtual seed markers without blocking the control loop.
Lab 5	Camera calibration, AprilTag, TF2	*Background:* the AprilTag on top of your robot is how the God Node finds you. You don't write this code, but you should know it's there.
Lab 6	OpenCV, HSV, circularity filter	*Background:* the God Node uses this to find weed balls for Phase 2. You consume the output.
Lab 7	Pure Pursuit, LERP breadcrumbs, headland arcs	The seeding coverage path. The single most important lab for the capstone.
Lab 8	SSH, odometry, Ackermann kinematics	Deploying your code to the Pi; understanding why turns must be arcs.
Lab 9	God Node bridge, `ROS_DOMAIN_ID`, DDS	Subscribing to `/tractor/gps`; running your controller on your laptop.
Lab 10	Farm Manager arbiter, Pure Pursuit, keyboard deadman	The complete safety stack. This *is* your capstone framework.

Table 1: Curriculum-to-capstone mapping.

Architecture: The MVP Strategy

Do **not** write the whole capstone as one monolithic node. Do **not** attempt to debug everything simultaneously on demo day. Follow this order:

Step	Test	What it proves
1	Drive forward exactly 1 m and stop.	Linear control + GPS works.
2	Turn 90° as an arc and stop.	Ackermann-aware angular control works.
3	Drive to a single God-Node-published waypoint without wobbling.	Pure Pursuit is tuned.
4	Drive a two-row snake pattern.	Path generation and headland turn work.
5	Publish `/seed_drop` manually from the command line and confirm a marker appears in the instructor's RViz.	Your topic name and message type are correct.
6	Fire `/seed_drop` from inside your Mission Runner every time the robot has travelled 10 cm, while driving straight.	Distance-based triggering works.
7	Drive the full snake pattern with seed drops firing along the way.	Seed logic and path-follow work together.
8	**Phase 1 on the live arena. This is the capstone.**	**You pass.**
9	(Phase 2) Subscribe to `/weed_locations` and visit each weed in order.	State machine handles mode switches.
10	(Phase 3) Add wet-zone detour.	Dynamic re-planning works.

Table 2: Demo-day order of operations. Reach Step 4 **before** arriving on demo day. Step 8 is the bar to clear.

The State Machine (Lab 10 applied)

Manage transitions with a Python `Enum`, not a tangle of booleans:

```
from enum import Enum

class MissionState(Enum):
    IDLE = 0
    SEEDING_NAV = 1
    SEEDING_DONE = 2
    # Stretch goals:
    HUNTING_WEED = 3
    SPRAYING = 4
    HARVEST_NAV = 5
    AVOIDING_WET_ZONE = 6
    DONE = 7
```

When you read your log file in the future, `state=SEEDING_NAV` is clear whereas `state=True` is a riddle.

Safety: The Deadman Switch

This is the only rule with no exceptions:

Non-Negotiable Safety Rule

If the spacebar is not held, `/cmd_vel` must be zero. Period.
The autopilot does not get a vote. The navigation algorithm does not get a vote. If the human releases the spacebar (intentionally, by accident, or because they sneezed) the motors stop within 50 ms.

You built the Farm Manager arbiter in **Lab 10** that enforces this. The structure: every controller publishes its *intent* to `/auto/cmd_vel`, and only the Farm Manager publishes to `/cmd_vel`. The deadman check runs on a **20 Hz timer**, not in the keyboard callback, because callbacks can be sparse and the safety check has to fire even when no keys are being pressed.

Figure 3: Only the Farm Manager writes `/cmd_vel`. Every other source must publish intent and hope the arbiter agrees.

Demo Day: How Arena Time Works

The arena is shared. You get the floor in 5-minute chunks, but if nobody is waiting, you can stay in the arena as long as you want. As soon as another group is ready and queued, finish your current attempt and rotate out.

The queue rules:

- One group in the arena at a time.
- If nobody is queued behind you, take your time, debug, retry.
- If someone is queued, your slot ends at 5 minutes (or sooner if you crash out / give up).
- Between turns, you can keep coding at your desk. Push fixes, rebuild, queue back up.

Each time you take the floor:

1. **Confirm you see the God Node.** In a fresh terminal:

   ```
   ros2 topic echo /tractor/gps
   ```

   Pose messages should stream. If you see nothing, your `ROS_DOMAIN_ID` is probably wrong. Ctrl-C once you're satisfied.

2. **Sanity check the deadman.** Robot on blocks if you can. Launch your stack:

   ```
   ros2 launch lab10 farm.launch.py
   ```

   In another terminal, fake an autonomous command:

```
ros2 topic pub /auto/cmd_vel geometry_msgs/msg/Twist \
  "linear: {x: 0.1}, angular: {z: 0.0}" --rate 10
```

In a third terminal, watch the final output:

```
ros2 topic echo /cmd_vel
```

Spacebar released: `/cmd_vel` is zero. **Spacebar held:** `/cmd_vel` matches the fake command. **Release the spacebar:** `/cmd_vel` snaps to zero. If any of these fail, do not put the robot on the floor.

3. **Manually drive the seed-drop pipeline once** to confirm RViz sees you. With your stack still running, publish a single seed-drop trigger:

   ```
   ros2 topic pub /seed_drop std_msgs/Empty "{}" --once
   ```

   A green marker should appear in the instructor's RViz at wherever the robot is sitting. If it doesn't, your topic name or message type is wrong; fix that before running the mission.

4. **Attempt the mission.** Phase 1 first. If Phase 1 works and you have time, try Phase 2.

5. **Release the spacebar the instant anything looks wrong.** Better to abort and re-queue than crash into another group's setup.

Pro Tip: Use Your Partner's Laptop

The commands above can be run from *any* laptop on the same `ROS_DOMAIN_ID`, not just the one running your stack. Have your partner sit next to you with their laptop and run the test publishers from there while you hold the spacebar on yours. This is the magic of ROS 2's underlying network layer (DDS): any node, anywhere on the network, can publish to any topic. Your two laptops are part of the same robot system.
This is also how the instructor's God Node, your laptop, and the robot all talk to each other without ever being plugged into the same machine.

Strategy Tip: Quick Attempts Beat Long Sessions

You will learn more from five short attempts with code tweaks in between than from one long attempt where you keep adjusting on the fly. Each time something doesn't work in the arena, take notes, go back to your laptop, fix one thing, and re-queue. The robot does not care how many times you tried.

What Success Looks Like

There is no grade for this. This is a workshop, not a course. Success looks like:

- Your robot drove a path autonomously, even if it was wobbly.
- Your deadman worked when you released it.
- You can point at the seeds in RViz.

- You learned something this month that you didn't know in April.

If you finished the labs, that's the win. Getting a month of ROS 2 into your head (while also fighting Docker Desktop, Windows path weirdness, and whatever else your laptop threw at you) is no small thing. Phase 1 is an achievement on top of that. Phase 2 and Phase 3 are extra credit for anyone who has time and energy. If your robot crashes into the wall on the first attempt and then you debug it and get a better run next time, that is also a win.

The real takeaway is not a working tractor. It's that you now know what it feels like to design, write, and deploy a real robotics stack. That experience will carry into whatever you build next.

Closing

A month ago you wrote a 30-line Python node that drove a turtle in a square. Today you have everything you need to deploy a real, autonomous, human-supervised agricultural robot. The control architecture you'll use this week is the same pattern running on PTX Trimble tractors. Same idea, different scale.

The work is done. You wrote it lab by lab. Now you're just plugging it together.